Recap

- Tag 2

Das Iterator-Protokoll

- Python löst dies durch "Lazy Evaluation" (verzögerte Auswertung).
- Ein Iterator ist ein Objekt, das zwei Methoden implementiert:
  1. `__iter__()`
     - Muss den Iterator selbst zurückgeben (`return self`).
     - Wird (implizit) von `for`-Schleifen aufgerufen.
  2. `__next__()`
     - Gibt das **nächste** Element der Sequenz zurück.
     - Wenn keine Elemente mehr vorhanden sind, löst es die `StopIteration` Exception aus.

Higher-Order-Functions

- MAP, FILTER, REDUCE

```
numbers = [1, 2, 3, 4]

# MAP 1:1
doubled_iterator = map(lambda x: x * 2, numbers)

# FILTER N:M
even_iterator = filter(lambda x: x % 2 == 0, numbers)

# REDUCE N:1
total = reduce(lambda acc, el: acc + el, numbers)
```

Generatoren: Funktionen

```
# Generator Funktion
def csv_reader(file_name):
  for row in open(file_name, 'r'):
    yield row.split(';')


# Verwendung der Funktion
for line in csv_reader('textfile.txt'):
  print(line)
```

Decorator

```
# 1. Der DECORATOR
def timer_decorator(func_to_wrap):
  # 2. Der WRAPPER
  def wrapper_function():
    start_time = time.perf_counter()

    func_to_wrap()  # 3. AUFRUF DER ORIGINALFUNKTION

    end_time = time.perf_counter()
    print(f"{func_to_wrap.__name__} dauerte {end_time - start_time:.4f}s")

  # 4. Der Decorator gibt den Wrapper zurück
  return wrapper_function


def process_data():
  """Eine Funktion, die dekoriert werden soll."""
  print("Starte Datenverarbeitung...")
  _ = [x ** 2 for x in range(10_000_000)]


# 5. Manuelle "Dekoration"
# Wir ersetzen die alte Funktion durch die neue Wrapper-Funktion
process_data = timer_decorator(process_data)

# 6. Aufruf des Wrappers
process_data()
```

Das re-Modul

```python
import re

text = "Der Preis ist 100€."

# re.search() findet das ERSTE Vorkommen im gesamten String
# r"\d+" bedeutet "eine oder mehrere Ziffern (\d)"
match = re.search(r"\d+", text)

if match:
    print(f"Treffer gefunden: {match.group(0)}")
    # Output: Treffer gefunden: 100
```

Type Annotations

- mit Python

Type Annotations

Python hat ein dynamisches Typsystem.

- Ab Python Version 3.5 wurden Type Hints eingebaut. Diese Hints werden zur Laufzeit ignoriert und sind nur eine Zusatzinformation für den Programmierer

- Tools können die Type Hints überprüfen und Warnungen ausgeben. Z. B. mypy, pyright (in VS Code: Pylance) oder PyCharm

- Ab Python Version 3.12 wurden folgende Dinge erweitert:
  - Definition von Typen mittels "`type`"
  - Definition von Generischen Funktionen/Klassen
  - Dekorator `@override` aus `typing` für überschriebene Methoden (PEP 698)

- Ab Python 3.13: Standardwerte für Typparameter (PEP 696), z. B. `class Box[T = int]`

Type Annotations

```
type IntOrFloat = int | float


def summe_lim(a: IntOrFloat, b: IntOrFloat) -> IntOrFloat:
  return a + b


def summe[T](a: T, b: T) -> T:
  return a + b


print(summe_lim(2, 3))
print(summe_lim(2.2, 3.3))
print(summe_lim("A", "B"))  # Fehler
print(summe(2, 3))
print(summe(2.3, 3.4))
print(summe("A", "B"))
```

Modul typing

- `list`, `tuple`, `dict` und `collections.abc` direkt verwenden, `typing` nur für Sonderformen

```
from collections.abc import Iterable

def first[T](items: Iterable[T]) -> T:
  for item in items:
    return item
  raise ValueError("Keine Elemente")

type MyTuple = tuple[str, int]
type MyTuple1[T] = tuple[str, T]
type MyVector = list[float]
```

Wichtige Typen aus typing

- `Union [X, Y]` oder `X | Y`
  - Entweder X oder Y
- `Optional[X]` oder `X | None`
  - Entweder X oder None
- `Any`
  - Wildcard. Schaltet Typchecker effektiv aus
- `Callable`
  - Beschreibt eine Funktion

Lab 10

Type Annotations & typing

DocString

- mit Python

Was ist ein DocString?

- String-Literal, das als **allererste Anweisung** in einem Modul, einer Klasse, einer Funktion oder einer Methode steht.
- **#** Kommentare: Sind für Entwickler (Maintainer). Sie erklären das **warum** und **wie** der Implementierung.
- **"""** Docstrings: Sind für Benutzer (Consumer). Sie erklären das **was** – wie man die Funktion benutzt (die API).

```
def my_function():
    """
    DAS ist der Docstring.
    Er wird zum __doc__-Attribut.
    """

    # DAS ist nur ein Kommentar.
    # Er wird vom Interpreter ignoriert.
    pass
```

Wer liest Docstrings?

Gute Docstrings sind keine "nette Geste", sondern eine technische Notwendigkeit für Tools.

1. Die `help()` -Funktion (Built-in)
2. IDEs & Editoren (z.B. VS Code, PyCharm)
3. Dokumentations-Generatoren (z.B. Sphinx)
4. KI-Agenten

Der Standard: *PEP 257*

Einzeilige Docstrings (One-Liners) für offensichtliche Funktionen. Endet mit einem Punkt.

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

Mehrzeilige Docstrings für komplexe Funktionen.

```
def complex_function(arg1, arg2):
    """Zusammenfassung in einem Satz (imperativ).

    Eine Leerzeile trennt die Zusammenfassung vom Detail.

    Hier folgt eine detailliertere Beschreibung der Logik,
    der Randbedingungen oder der Nutzungsszenarien.
    """
    pass
```

Fortgeschrittene Formate

Damit Tools Argumente, Rückgabewerte und Fehler verstehen, hat sich ein Standard-Layout etabliert. Die zwei populärsten sind Google Style und NumPy Style.

```python
def get_item(item_id: int, include_details: bool = False) -> dict:
    """Holt einen Artikel aus der Datenbank basierend auf seiner ID.

    Args:
        item_id (int): Die primäre ID des Artikels, der gesucht wird.
        include_details (bool, optional):
            Wenn True, werden zusätzliche Details mitgeladen.
            Defaults to False.

    Returns:
        dict: Ein Dictionary mit den Artikeldaten.

    Raises:
        ItemNotFoundError: Wenn keine Artikel mit der 'item_id'
                           gefunden wurde.
    """
    if item_id == 404:
      raise ItemNotFoundError(f"Item {item_id} nicht gefunden.")

    data = {"id": item_id, "name": "Beispiel"}
    if include_details:
      data["details"] = "Weitere Infos..."
    return data
```

Struktur großer Programme

- mit Python

Code-Struktur für große Projekte

- Für ein Skript oder eine kleine App mag das ausreichen

```
my_project/
├── main.py
├── utils.py
├── models.py
├── test_main.py
└── requirements.txt
```

Das src-Layout

Trennen Sie *installierbaren Code* (`src`) vom Rest des Projekts (Tests, Doku, Konfig).

```
my_project/
├── .venv/
├── .git/
├── pyproject.toml   # Dependencies
├── README.md
├── src/             # <- HIER lebt der Code
│   └── my_app/      # Das eigentliche Python-Package
│       ├── __init__.py
│       ├── main.py    # (z.B. der App-Einstiegspunkt)
│       ├── users/
│       └── payments/
├── tests/           # <- Tests sind klar getrennt
│   ├── test_users.py
│   └── test_payments.py
└── docs/
```

Modulstruktur nach *Typ*

Antipattern:

- Führt zu hoher Kopplung. `views.py` muss `models.py` und `utils.py` importieren. Eine Änderung in `models.py` bricht potenziell alles.

```
src/my_app/
   ├── models.py     # (Alle 20 DB-Modelle)
   ├── views.py      # (Alle 50 API-Endpunkte)
   ├── serializers.py
   └── utils.py
```

Modulstruktur nach *Feature*

- Best practise nach *Domain Driven Design*
- Jedes Feature ist ein eigenes Mini-Paket. Änderungen am "payment"-Modell beeinträchtigen das "user"-Modul nicht.

```
src/my_app/
    ├── __init__.py
    ├── core/               # (z.B. DB-Verbindung, Auth)
    ├── users/              # FEATURE 1
    │   ├── __init__.py
    │   ├── models.py       # (Nur User-Modelle)
    │   ├── views.py        # (Nur User-Endpunkte)
    │   └── services.py     # (Business-Logik)
    └── payments/           # FEATURE 2
        ├── __init__.py
        ├── models.py
        ├── views.py
        └── services.py
```

Konfigurationsmanagement

1. Statische Konfiguration (Projekt-Definition `pyproject.toml`)
   - Was: Abhängigkeiten, Projekt-Name, Build-Tools.
   - Wer: Wird vom **Entwickler** definiert und im Repo gespeichert.
2. Dynamische Konfiguration (Environment-Variablen `.env`)
   - Was: Secrets (API-Keys, DB-Passwörter), Host-Namen, `DEBUG`-Flag
   - Wer: Wird von der **Umgebung** (Server, Docker, Entwickler-Maschine) bereitgestellt.

Konfigurationsmanagement

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    # Liest automatisch aus Environment-Variablen
    # .env nur mit dieser Zeile:
    model_config = SettingsConfigDict(env_file=".env")
    db_url: str
    debug_mode: bool = False

# App lädt Settings *nur* aus der Umgebung bzw. .env
settings = Settings()
db = connect(settings.db_url)
```

UV

- mit Python

Warum UV?

Vor uv brauchte man für ein Projekt folgende Tools:

- Umgebung: `python -m venv` oder `virtualenv`
- Installation: `pip`
- Abhängigkeiten (statisch): `requirements.txt`
- Locking: `pip-tools` (z.B. `pip-compile`)
- Projekt-Metadaten: `setup.py` / `pyproject.toml`
- Ausführung: `source .venv/bin/activate`

UV Konfiguration: *pyproject.toml*

`requirements.txt` wird nicht mehr für die **Definition** von Abhängigkeiten benötigt. Alles lebt in `pyproject.toml`.

```
[project]
name = "my_app"
version = "0.1.0"
requires-python = ">=3.10"
# 1. Haupt-Abhängigkeiten (ersetzt requirements.txt)
dependencies = [
    "fastapi>=0.100.0",
    "sqlalchemy[asyncio]",
]
# 2. Entwicklungs-Abhängigkeiten (uv add --dev ...)
[dependency-groups]
dev = [
    "pytest",
    "pytest-mock",
    "mypy",
    "ruff",
]
```

UV: *Dependency Management*

Projekt anlegen (ersetzt `python -m venv`)

```
# .venv legt uv beim ersten sync/run an
$ uv init my_app
```

Abhängigkeiten installieren (ersetzt `pip install -r`)

```
# Installiert dependencies + Gruppe dev
$ uv sync
# Ohne Dev-Abhängigkeiten (z. B. Produktion)
$ uv sync --no-dev
```

Neue Pakete hinzufügen (ersetzt `pip install package`)

```
# Installiert 'requests' UND fügt es zu [project.dependencies] hinzu
$ uv add requests
```

UV: *Dependency Locking*

uv sync schreibt uv.lock und installiert exakt die gelockten Versionen, auch wenn es neuere gibt. Upgrades nur bewusst:

```
# Auf neuere Versionen heben
$ uv lock --upgrade
```

```
# Im Dockerfile:
COPY pyproject.toml uv.lock ./
# Installiert NUR die exakten Versionen aus der Lock-Datei
# --locked: Abbruch, falls uv.lock veraltet ist
# --no-dev: ohne Dev-Abhängigkeiten
RUN uv sync --locked --no-dev
```

UV: *Tests ausführen*

`uv run` vereinfacht CI-Pipelines drastisch, da keine `activate`-Skripte benötigt werden.

```
# 1. Sicherstellen, dass die Umgebung + Dev-Gruppe installiert sind
$ uv sync

# 2. Tests ausführen (ohne 'source activate')
$ uv run pytest

# 3. Linter ausführen
$ uv run ruff check .
$ uv run mypy src/
```

UV: *Build & Install*

uv kann Ihr Projekt (definiert in `pyproject.toml`) in ein verteilbares Paket (Wheel) bauen.

```
$ uv build
# Ergebnis: dist/my_app-0.1.0-py3-none-any.whl
```

Lokal "installieren" (ersetzt `pip install -e .`)
`uv sync` installiert das Projekt selbst editable

```
$ uv sync
```

UV: Zusammenfassung

1. Lokale Entwicklung:
   - `uv init`
   - `uv add` / `uv add --dev`
   - `uv run`
2. CI-Pipeline (z.B. GitHub Actions)
   - `uv sync --locked`
   - `uv run mypy`
   - `uv run pytest`
3. Build-Artefakt erstellen (optional, für PyPI)
   - `uv build`
   - `uv publish`
4. Deployment (z.B. Docker-Image)
   - `uv sync --locked --no-dev`
   - oder: `uv export --no-dev -o requirements.txt`

Demo

DocString, Projektstruktur, uv

Requests

- mit Python

Warum requests?

- Pythons eingebaute `urllib` ist mächtig, aber umständlich

```
import urllib.request
  import json

  # Umständlich, nicht intuitiv, fehleranfällig
  try:
      with urllib.request.urlopen("https://api.example.com") as response:
          data = json.loads(response.read().decode())
  except urllib.error.HTTPError as e:
      print(f"Fehler: {e.code}")
```

requests: GET

- CRUD: POST/GET/PUT/DELETE

```python
import requests

url = "https://api.github.com/users/octocat"
response = requests.get(url)

# --- Das 'Response'-Objekt ---

# 1. Status-Code
print(f"Status: {response.status_code}")

# 2. Inhalt (Roh-Text, z.B. für HTML)
print(response.text)

# 3. Inhalt (Bytes, z.B. für Bilder)
print(response.content)

# 4. JSON (Der häufigste Fall für APIs)
# Parst den JSON-Text automatisch in ein Python-dict
data = response.json()
print(f"Name: {data['name']}")
```

requests: POST

- Server State verändern

```
# Sendet als application/x-www-form-urlencoded
payload = {'username': 'alice', 'pass': '123'}
r = requests.post("https://httpbin.org/post", data=payload)


# Setzt den `Content-Type: application/json` Header automatisch
payload = {'username': 'alice', 'role': 'admin'}
r = requests.post("https://api.example.com/users", json=payload)

print(r.json())
```

requests: Error Handling

- requests löst bei 4xx- oder 5xx-Fehlern **keine** Exception aus!

```
from requests.exceptions import HTTPError

try:
  response = requests.get("https://api.github.com/invalid-url")

  # Erzwinge eine Exception bei schlechtem Status
  response.raise_for_status()

  data = response.json()

except HTTPError as http_err:
  print(f"HTTP-Fehler aufgetreten: {http_err}")
except Exception as err:
  print(f"Anderer Fehler: {err}")
```

requests: Timeouts

- Standardmäßig wartet `requests` **unbegrenzt** auf eine Antwort.

```
from requests.exceptions import Timeout

try:
  response = requests.get(
    "https://api.example.com/slow-endpoint",
    timeout=5.0
  )

except Timeout:
  print("Der Request ist abgelaufen (Timeout).")
```

requests: Sessions

- Eine `Session` verwaltet **Cookies** und nutzt **Connection-Pooling.**

```python
with requests.Session() as s:

  # Login (Server setzt ein Cookie)
  login_data = {'username': 'user', 'pass': 'pass'}
  s.post("https://example.com/login", data=login_data)

  # Nächste Anfrage (sendet das Cookie automatisch mit)
  # Wir sind jetzt eingeloggt
  response = s.get("https://example.com/dashboard")

  print(response.text)
```

requests: Headers & Auth

- Wie sendet man einen API-Key (z.B. Bearer-Token)?

```
api_key = "my-secret-token-123"

headers = {
  "Authorization": f"Bearer {api_key}",
  "User-Agent": "MyAwesomeApp/1.0"
}

with requests.Session() as s:
  s.headers.update(headers)

  # Jede Anfrage von 's' sendet diese Header jetzt
  r = s.get("https://api.example.com/secure/data")
```

Lab 12

requests

Daten-Analyse mit Pandas

- mit Python

Phyton-Listen vs. “Daten”

- Python ist flexibel, aber Standard-Listen oder Dictionaries sind für "tabellarische" Daten ungeeignet.
- **Keine Vektorisierung**: Eine Operation auf 1 Million Listenelemente erfordert eine langsame Python-`for`-Schleife.
- **Keine Labels**: `data[0][4]` – Was bedeutet das? Spalten haben keine Namen.
- **Mühsames Handling**: Operationen wie Gruppieren, Zusammenführen (Joins) oder der Umgang mit fehlenden Werten (`None`) müssen manuell implementiert werden.
- **Pandas-Lösung**: Bietet Hochleistungs-Datenstrukturen (basierend auf `NumPy`), die speziell für tabellarische Datenanalyse optimiert sind.

Die Kern-Datenstrukturen

`Series` (1D)

- Eine einzelne Spalte (ein 1D-Array).
- Hat einen **Index** (Labels für jede Zeile).
- `s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])`

`DataFrame` (2D)

- Die Haupt-Datenstruktur (eine Tabelle).
- Vgl. Excel-Sheet oder eine SQL-Tabelle.
- Ist im Grunde eine **Sammlung von `Series`-Objekten**, die sich einen Index teilen.

Daten einlesen (pd.read_...)

- Pandas kann fast jedes gängige Datenformat direkt in einen `DataFrame` laden.

```
import pandas as pd

# Liest eine CSV-Datei und leitet Spaltentypen/Header ab
df = pd.read_csv("sales_data.csv")

# Andere gängige Formate:
# df = pd.read_excel("report.xlsx")
# df = pd.read_json("api_response.json")
# df = pd.read_sql("SELECT * FROM users", db_connection)
```

Das Ergebnis `df` ist ein `DataFrame`-Objekt.

Erste Inspektion

Nach dem Laden müssen Sie die Daten verstehen. Hilfreiche Befehle:

- **`df.head()`** Zeigt die **ersten 5 Zeilen**. (Prüft, ob das Laden korrekt aussieht).
- **`df.info()`** **WICHTIG**: Zeigt Spaltennamen, die Anzahl der Nicht-Null-Werte und den Datentyp (dtype) jeder Spalte. (Zeigt fehlende Daten\!).
- **`df.describe()`** Generiert eine statistische Zusammenfassung (Mittelwert, Median, Min/Max, Quartile) für **numerische** Spalten.

Inspektion

```
df.info()
# <class 'pandas.DataFrame'>
# RangeIndex: 1000 entries, 0 to 999
# Data columns (total 3 columns):
#  #   Column     Non-Null Count  Dtype
# ---  ------     --------------  -----
#  0   UserID     1000 non-null   int64
#  1   Product    950 non-null    str     <-- 50 fehlende Werte!
#  2   Price      1000 non-null   float64
```

Selektion: Spalten auswählen Indexing

- Das Auswählen von Spalten ist intuitiv.

```
# Syntax: df['Spaltenname']

# 1. Eine einzelne Spalte auswählen -> Ergibt eine Series
users = df['UserID']
# print(type(users)) # <class 'pandas.Series'>

# 2. Mehrere Spalten auswählen -> Ergibt einen DataFrame
# (Beachten Sie die doppelte Klammer: df[[list_of_names]])
user_data = df[['UserID', 'Product']]
# print(type(user_data)) # <class 'pandas.DataFrame'>
```

Zeilen auswählen (.loc vs. .iloc)

Die []-Syntax ist für Zeilen zweideutig. Verwenden Sie **immer** `.loc` oder `.iloc` für die Zeilenauswahl.

- `.loc` (Label)

Wählt Zeilen basierend auf dem **Index-Label**

- `df.loc[5]`

Holt die Zeile mit dem Index-Label `5` (kann auch ein String sein, z.B. `'a'`).

- `df.loc[5:10]`

Holt Zeilen `5` bis `10`

- `.iloc` (Integer-Position)

Wählt Zeilen basierend auf der **numerischen Position** (wie bei Python-Listen).

- `df.iloc[5]`

Holt die 6. Zeile (0-basiert).

- `df.iloc[5:10]`

Holt Position `5` bis `9`

Filtern (Boolean Indexing)

- Dies ist der mächtigste Weg, um Daten basierend auf Bedingungen auszuwählen.

- Konzept (Die "Maske"):

1. Erstellen Sie eine `Series` von `True/False`-Werten (die "Maske").
2. Übergeben Sie diese Maske an den `DataFrame` (`df[...]`).

Filtern (Boolean Indexing)

```
df = pd.DataFrame({'age': [22, 45, 30], 'city': ['BER', 'NYC',
'BER']})

# 1. Die Bedingung (erzeugt eine Series [False, True, False])
mask = df['age'] > 40

# 2. Anwenden der Maske
# Pandas gibt nur die Zeilen zurück, bei denen die Maske True war.
old_users = df[mask]
# print(old_users) # Zeigt nur den 45-jährigen User

# --- Kombinierte Bedingungen ---
# & (und), | (oder). Klammern () sind zwingend!
mask_complex = (df['age'] > 25) & (df['city'] == 'BER')

users_berlin = df[mask_complex]
# print(users_berlin) # Zeigt nur den 30-jährigen User
```

groupby (Split-Apply-Combine)

- groupby ist das wichtigste Werkzeug für fast jede Datenanalyse.
- Konzept :

1. **Split**: Teilt den DataFrame in Gruppen basierend auf einem Kriterium (z.B. "Stadt")
2. **Apply**: Wendet eine Aggregations-Funktion auf jede Gruppe an (z.B. `sum()`, `mean()`, `count()`)
3. **Combine**: Fügt die Ergebnisse zu einem neuen DataFrame zusammen.

groupby (Split-Apply-Combine)

```
# Frage: "Was ist der Durchschnittspreis pro Produktkategorie?"
# df hat Spalten: 'Category', 'Price'

# 1. Split (nach 'Category')
# 2. Apply (mean() auf 'Price')
# 3. Combine
avg_price_per_category = df.groupby('Category')['Price'].mean()

# Frage: "Wie viele Sales pro Stadt und Kategorie?"
sales_count = df.groupby(['City', 'Category'])['SalesID'].count()
```

Key Takeaways

- **DataFrame (2D)**: Die primäre Datenstruktur. Eine Tabelle.
- **Series (1D)**: Eine einzelne Spalte.
- **I/O**: `pd.read_csv()` (und andere) sind der Startpunkt.
- **Inspektion**: `df.head()`, `df.info()`, `df.describe()` sind obligatorisch.
- **Selektion**:
  1. Spalten: `df[['Col_A', 'Col_B']]`
  2. Zeilen: `.loc` (Label) oder `.iloc` (Position).
- **Filtern**: Boolean Indexing (Masking) ist der Standard (`df[df['age'] > 30]`).
- **Analyse** (Kern): `groupby()` (Split-Apply-Combine) ist das mächtigste Werkzeug zur Aggregation von Daten.

Lab 13

Daten-Analyse mit Pandas

Logging

- mit Python

Warum `print()` nicht reicht

In der Entwicklung ist print() okay – im Betrieb (Webserver, Datenpipeline, Hintergrunddienst) nicht.

- **Keine Filterung**: alles oder nichts, kein „nur Fehler anzeigen“.
- **Kein Kontext**: keine Zeit, kein Modul, kein Level.
- **Festes Ziel**: immer nach `stdout`. Datei, Konsole oder Log-Server lassen sich nicht umstellen.

- **Lösung**: Das **`logging`-Modul**. Es trennt das **Erzeugen** einer Meldung (im Code) von ihrer **Verarbeitung** (Konfiguration: Level, Format, Ziel).

Basics: Log Levels

- Das Level wird in der **Konfiguration** gesetzt, nicht im Code. Sie ändern die Ausführlichkeit also, ohne den Code anzufassen.

Level	Logging – Funktion	Nutzung
DEBUG	`logging.debug()`	Details zur Diagnose (z.B. Variablen-Werte)
INFO	`logging.info()`	Normaler Ablauf („Service gestartet“)
WARNING	`logging.warning()`	Unerwartet, App läuft weiter („Nicht genügend Deckung“)
ERROR	`logging.error()`	Eine Operation ist fehlgeschlagen
CRITICAL	`logging.critical()`	Die Anwendung kann nicht weiterlaufen

Basics: logging.basicConfig()

```
import logging

# Konfiguration muss VOR dem ersten logging-Aufruf erfolgen!
logging.basicConfig(
    level=logging.DEBUG, # Ab welchem Level anzeigen?
    format="%(asctime)s - %(levelname)s - %(message)s",
    filename="app.log" # Optional: in eine Datei loggen
)

logging.debug("Detail-Info")
logging.info("Service gestartet")
logging.warning("Passwort ist schwach")
```

- Konfiguriert den **Root-Logger**, und zwar **nur beim ersten Aufruf**. Hat der Root-Logger schon einen Handler, passiert nichts (außer mit `force=True`).

Pro Modul ein Logger: getLogger(__name__)

`__name__` im Format zeigt, woher die Meldung kommt (`account`, `__main__`, `urllib3.connectionpool` …).

- **Feinsteuerung**: Logger bilden eine Hierarchie (`my_app` → `my_app.db`). Sie können z. B. nur `my_app.db` auf `DEBUG` stellen.
- **%-Platzhalter statt f-String**: Der Text wird nur formatiert, wenn die Meldung wirklich ausgegeben wird.

```python
# account.py
import logging
log = logging.getLogger(__name__)  # Name = Modulname, hier "account"
def withdraw(account_number: str, balance: float, amount: float) -> bool:
    if amount > balance:
        log.warning("Nicht genügend Deckung für Konto %s", account_number)
        return False
    log.debug("Abhebung von %.2f EUR von %s", amount, account_number)
    return True
```

App konfiguriert, Library loggt

Library / Modul (account.py, requests)	Applikation (main.py)
`log = logging.getLogger(__name__)`	konfiguriert einmal beim Start (`basicConfig / dictConfig`)
ruft nur `log.info()`, `log.warning()` … auf	legt Level, Format und Ziel fest (Konsole, Datei, JSON)
nie `basicConfig()`, keine Handler außer `NullHandler`	entscheidet auch über fremde Logger (z. B. `urllib3` leiser stellen)

Exceptions korrekt loggen

```
import logging

logging.basicConfig(level=logging.DEBUG)
log = logging.getLogger(__name__)

try:
    risk = 500.0 / 0
except ZeroDivisionError as e:
    # FALSCH: nur "division by zero" – kein Stack Trace, keine Zeilennummer
    log.error(f"Ein Fehler ist aufgetreten: {e}")

    # RICHTIG: Level ERROR + vollständiger Stack Trace
    log.exception("Schwerwiegender Fehler bei der Risikoberechnung")

    # Alternative mit anderem Level
    log.warning("Risikoberechnung fehlgeschlagen", exc_info=True)
```

logging.config.dictConfig()

```
import logging.config

LOGGING_CONFIG = {
    "version": 1,
    "disable_existing_loggers": False,  # sonst werden schon importierte Modul-Logger stumm
    "formatters": {  # WIE sieht eine Zeile aus?
        "default": {"format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s"},
    },
    "handlers": {  # WOHIN geht sie?
        "console": {"class": "logging.StreamHandler", "formatter": "default"},
    },
    "loggers": {  # WER loggt ab welchem Level?
        "": {"level": "INFO", "handlers": ["console"]},  # Root: gilt für alle
        "account": {"level": "DEBUG"},                    # unser Modul ausführlich
        "urllib3": {"level": "WARNING"},                  # HTTP-Details von requests leise
    },
}

logging.config.dictConfig(LOGGING_CONFIG)  # einmal, beim Start der App
```

Structured Logging (JSON)

Statt einer Textzeile wird pro Meldung ein JSON-Objekt geloggt: maschinenlesbar für ELK, Datadog, Loki …

```
import logging.config

logging.config.dictConfig({
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "json": {
            "class": "pythonjsonlogger.json.JsonFormatter",  # nicht mehr .jsonlogger
            "format": "%(asctime)s %(name)s %(levelname)s %(message)s",
        },
    },
    "handlers": {
        "console_json": {"class": "logging.StreamHandler", "formatter": "json"},
    },
    "loggers": {"": {"level": "INFO", "handlers": ["console_json"]}},
})

log = logging.getLogger("app.users")
log.error("Login fehlgeschlagen", extra={"user_id": 123, "login_attempt": 5})
# {"asctime": "...", "name": "app.users", "levelname": "ERROR",
#  "message": "Login fehlgeschlagen", "user_id": 123, "login_attempt": 5}
```

Lab 14

Logging

Unit-Tests mit pytest

- mit Python

Warum Unit-Tests?

- Manuelles Testen (z.B. Skript starten, `print()`-Ausgaben prüfen) ist:
  - **Langsam**: Erfordert manuelle Interaktion.
  - **Fehleranfällig**: Man vergisst, Edge Cases zu prüfen.
  - **Nicht wiederholbar**: Ein Kollege kann den Test nicht exakt gleich ausführen.
  - **Blockierend**: Verhindert sicheres Refactoring.
- Jede Zeile Code, die Sie schreiben, ist eine potenzielle **Fehlerquelle** ("Liability").
- Tests sind das **Sicherheitsnetz** (Safety Net), das beweist, dass Ihr Code (und zukünftiges Refactoring) funktioniert.
- **Unit-Test**: Ein automatisierter Test, der die kleinste isolierte Einheit (eine Funktion oder Methode) prüft.

pytest vs. unittest

`unittest` ist in Python eingebaut, aber `pytest` ist der De-facto-Standard in der Industrie.

FEATURE	`unitest`	`pytest`
Boilerplate	**Hoch.** Man muss von `TestCase` erben.	**Keins.** Einfache `test_...` Funktionen.
Asserts	**Umständlich** (`self.assertEqual(a, b)`)	Nativ (`assert a == b`)
Fehler-meldung	**Limitiert** (`AssertionError: X != Y`)	Zeigt genau, was ungleich ist ("Magic Asserts")
Cleanup	`setUp()`, `tearDown()` Methoden	**Fixtures** (Flexibler, wiederverwendbar)

pytest vs. unittest

```
# unittest (Umständlich)
import unittest

class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)


# pytest (Pythonic, sauber)
# (Keine Imports, keine Klasse nötig)

def test_add():
    assert add(2, 3) == 5
```

Test Discovery & assert

Test Discovery

- Wie findet `pytest` Ihre Tests?
- Sucht nach Dateien namens `test_*.py` oder `*_test.py`.
- Innerhalb dieser Dateien, sucht nach:
  - Funktionen namens `test_*()`.
  - Klassen namens `Test*` (die kein `__init__` haben dürfen).

```python
def test_capitalize():
  assert "hello".capitalize() == "Hello"


def test_type_error():
  # Testet, ob der erwartete Fehler
auftritt
  with pytest.raises(TypeError):
    "string" + 5
```

Setup & Cleanup @pytest.fixture

- **Problem**: Viele Tests benötigen denselben Setup-Code (z.B. eine DB-Verbindung, ein User-Objekt, ein temporäres Verzeichnis).
- Lösung -> **Fixtures**: Eine Funktion, die mit `@pytest.fixture` markiert ist und Daten "bereitstellt" (via `return` oder `yield`).Tests "fordern" Fixtures an, indem sie sie als Argumente benennen (Dependency Injection).
- Trennt das Setup (die Fixture) sauber von der Test-Logik. Extrem wiederverwendbar.

Setup & Cleanup @pytest.fixture

```python
import pytest

@pytest.fixture
def empty_user_db():
    """Stellt eine leere In-Memory-DB bereit."""
    db = setup_database()
    yield db # 1. Test läuft hier...
    db.cleanup() # 2. ...Teardown (wird garantiert ausgeführt)

# --- Nutzung ---

def test_add_user(empty_user_db): # Fixture wird hier "injiziert"
    db = empty_user_db
    db.add_user("alice")
    assert db.count() == 1
```

Parametrisierung @pytest.mark.parametrize

- Wie testet man eine Funktion mit 10 verschiedenen Eingaben (Gültige Daten, ungültige Daten, `None`, 0, ...)?
- Schlechte Lösung: 10 fast identische `test_...`-Funktionen kopieren.
- Beste Lösung (DRY): `@pytest.mark.parametrize`
- Erlaubt es, **eine** Testfunktion mit **mehreren** Datensätzen laufen zu lassen.
- `pytest` führt 5 separate Tests aus. Wenn "invalid-mail" fehlschlägt, werden die anderen 4 trotzdem als "PASS" angezeigt.

Parametrisierung @pytest.mark.parametrize

```python
def is_valid_email(email):
  # ... Logik ...
  pass


@pytest.mark.parametrize("email_input, expected_result", [
  ("test@mail.com", True),  # Fall 1
  ("user.name@sub.domain.org", True),  # Fall 2
  ("invalid-mail", False),  # Fall 3 (Edge Case)
  ("", False),  # Fall 4 (Edge Case)
  (None, False),  # Fall 5 (Edge Case)
])
def test_email_validation(email_input, expected_result):
  assert is_valid_email(email_input) == expected_result
```

Isolation & Mocking (pytest-mock)

- **Problem**: Ein **Unit-Test** muss isoliert sein. Wenn `test_payment()` eine echte externe API (z.B. Stripe, PayPal) aufruft, ist das:
  - **Langsam** (Netzwerk-Latenz)
  - **Unzuverlässig** (Die API kann down sein)
  - **Teuer** (kann Geld kosten)
  - **Kein Unit-Test** (sondern ein Integrationstest).
- **Lösung: Mocking**
- Das Ersetzen von externen Abhängigkeiten (wie `requests.post`) durch "Attrappen" (Mocks).
- Das `pytest-mock` Plugin (stellt die `mocker`-Fixture bereit).

Isolation & Mocking (pytest-mock)

```python
def test_charge_user(mocker):  # 'mocker' Fixture anfordern

  # Ersetze 'requests.post' durch einen Mock,
  # der ein Fake-JSON zurückgibt
  mocker.patch("requests.post",
               return_value=MockResponse(status_code=200))

  # Führe den Test aus
  result = payment_service.charge_user("user_123", 100)

  # Teste *unseren* Code, nicht die API
  assert result == "SUCCESS"
```

Lab 15

Unit-Tests mit pytest

Datenbank

- mit Python

Warum Persistenz?

- In-Memory-Daten sind nach `exit()` verloren.
- Mehrere Programme / Server müssen sich dieselben Daten teilen.
- Datenbanken liefern wertvolle Garantien:
  - Atomare Transaktionen (ACID): Eine Überweisung gelingt ganz oder gar nicht.
  - Constraints (`UNIQUE`, `NOT NULL`, `FOREIGN KEY`) erzwingen Datenqualität auf Storage-Ebene.
  - Indizes für schnelle Suchen auch bei Millionen Zeilen.
- Für Lernen, Tests und kleine Apps: SQLite — die ganze DB liegt in einer einzigen Datei.

SQLite — die Datenbank im File

- Eine ganze relationale Datenbank in einer einzigen .db-Datei.
- Eingebettet: kein Server-Prozess, keine Konfiguration.
- Ideal für: Lernen, Unit-Tests, eingebettete Geräte, Apps mit einem einzigen Nutzer.
- Standard-Tooling:
  - Kommando-Zeile: sqlite3 bank.db
  - Python-Standardbibliothek: import sqlite3 — schon da, keine Installation.
- Produktion mit vielen parallelen Schreibern: lieber PostgreSQL / MySQL — aber das API ist (dank SQLAlchemy) fast dasselbe.

sqlite3

```python
import sqlite3

# 1. Verbinden (Datei wird erstellt, falls nicht vorhanden)
conn = sqlite3.connect("app.db")
cursor = conn.cursor()

# 2. Schema definieren
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        email TEXT
    )
   """
   )

# 3. Daten einfügen + Transaktion bestätigen
cursor.execute("INSERT INTO users (name) VALUES (?)", ("Alice",))
conn.commit()

# 4. Daten abfragen
for row in cursor.execute("SELECT id, name FROM users"):
    print(row)   # -> (1, 'Alice')

conn.close()
```

SQL-Injection & parameterisierte Queries

- Verlockung: den Wert direkt in den SQL-String basteln.
- Übergibt ein böser Nutzer `' OR 1=1 --`, liefert die Query plötzlich alle Zeilen.
- Richtig: Platzhalter verwenden — der Treiber escaped automatisch.
- Werte gehören NIEMALS per f-string oder + in einen SQL-String.

```
# GEFAHR – niemals so machen!
cursor.execute(f"SELECT * FROM users WHERE name = '{user_input}'")

# Richtig – Platzhalter, der Treiber escaped automatisch:
cursor.execute("SELECT * FROM users WHERE name = ?", (user_input,))

# In SQLAlchemy heißt der Platzhalter ":name"
conn.execute(text("SELECT * FROM users WHERE name = :n"), {"n": user_input})
```

Object-Relational Impedance Mismatch

- SQL denkt in Tabellen, Spalten, Tupeln.
- Python denkt in Objekten, Attributen, Methoden.
- Folge: viel manueller Konvertierungs-Code.
- Diese Übersetzung vermeiden wir später mit einem ORM (Object-Relational Mapper).

```
class User:
       def __init__(self, id, name):
           self.id = id
           self.name = name

   cursor.execute("SELECT id, name FROM users WHERE id = 1")
   row = cursor.fetchone()        # -> (1, 'Alice')   ein Tupel

   # Boilerplate: Tupel -> Objekt
   if row:
       user = User(id=row[0], name=row[1])
```

sqlalchemy

- Die de-facto-Datenbank-Bibliothek für Python.
- Arbeitet mit allen gängigen Datenbanken (SQLite, PostgreSQL, MySQL, Oracle, MS SQL …).
- Zwei Schichten, die man frei kombinieren kann:
  - Core — Engine, Connection, text(…)-SQL. Näher an SQL, kein ORM-Overhead.
  - ORM — Klassen werden auf Tabellen gemappt, Queries über select(MyClass).
- Vorteile gegenüber rohem sqlite3:
  - Datenbank-Portabilität: derselbe Code für SQLite (Test) und PostgreSQL (Produktion).
  - Connection-Pooling out-of-the-box.
  - Sicher per Default: parameterisierte Queries sind die einzige bequeme Variante.

sqlalchemy Core

- `create_engine(...)` baut noch keine Verbindung auf — erst `connect()` tut das.
- Connection-URL: `sqlite:///pfad.db`, `postgresql+psycopg://user:pw@host/db` usw.
- Wechseln der DB = Wechseln der URL. Der Python-Code ändert sich nicht.

```python
from sqlalchemy import create_engine, text

# Engine: zentraler Einstiegspunkt + Connection-Pool
engine = create_engine("sqlite:///bank.db")

# Verbindung holen, automatisch zurück in den Pool
with engine.connect() as conn:
    result = conn.execute(text("SELECT account_number, owner FROM accounts"))
    for row in result:
        print(row)
```

sqlalchemy Core — Parametrisierte Queries

- `text(...)` umhüllt rohes SQL, damit `sqlalchemy` es nicht als ORM-Statement missversteht.
- `.mappings()` macht dict-artige Rows — Spalten heißen wie in der DB.
- `.first()` holt die erste Zeile (oder None), `.all()` alle.

```
with engine.connect() as conn:
    # Parameter als Dict, Platzhalter als ":name"
    stmt = text(
        "SELECT account_number, owner, balance "
        "FROM accounts "
        "WHERE account_number = :number"
    )
    result = conn.execute(stmt, {"number": "AT001"})

    # .mappings() liefert dict-artige Rows (Zugriff per Spaltenname)
    row = result.mappings().first()
    print(row["owner"], row["balance"])
```

ORM — Modell-Definition

- Die Klasse IST die Tabelle, die Attribute SIND die Spalten — keine Tupel-Übersetzung mehr.
- `Mapped[type]` ist eine echte Type-Annotation
- Älterer Stil mit `Column(Integer, ...)` funktioniert noch, gilt aber als Legacy.

```
from sqlalchemy import ForeignKey
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class Account(Base):
    __tablename__ = "accounts"

    id: Mapped[int]              = mapped_column(primary_key=True)
    account_number: Mapped[str]  = mapped_column(unique=True)
    owner: Mapped[str]
    balance: Mapped[float]

    transactions: Mapped[list["Transaction"]] = relationship(
        back_populates="account",
    )
```

ORM — Session & Lebenszyklus

- Die Session ist die ORM-Variante der Connection — sie koordiniert eine Unit of Work.
- `add(...)` markiert das Objekt; das SQL entsteht erst bei `commit()` (oder `flush()`).
- `commit()` schreibt alle Änderungen in einer Transaktion — alles oder nichts.
- Verlässt man den with-Block ohne `commit()`, wird automatisch `rollback()` aufgerufen.

```
from sqlalchemy.orm import Session

with Session(engine) as session:
    new_account = Account(
        account_number="AT999",
        owner="Neuer Kunde",
        balance=100.0,
    )
    session.add(new_account)     # bei der Session registrieren
    session.commit()             # SQL INSERT ausführen + bestätigen
```

ORM — select()

- `select(Account)` baut ein Statement, `session.execute(...)` führt aus.
- `.scalars()` macht aus dem 1-Spalten-Result echte Account-Objekte.
- `.scalar_one_or_none()` für Einzeltreffer — wirft nicht, wenn nichts gefunden wird.

```
from sqlalchemy import select

with Session(engine) as session:
    # alle Konten
    stmt = select(Account).order_by(Account.account_number)

    for account in session.execute(stmt).scalars():
        print(account.account_number, account.owner)

    # ein Konto per Bedingung
    stmt = select(Account).where(Account.account_number == "AT001")
    account = session.execute(stmt).scalar_one_or_none()

    if account:
        print(account.balance)
```

ORM — Beziehungen

- `ForeignKey("accounts.id")` ist der DB-Constraint, `relationship(...)` die Objekt-Sicht.
- back_populates sorgt für eine konsistente Gegenrichtung — beide Seiten kennen einander.
- Statt JOIN-SQL schreibt man `account.transactions` — SQLAlchemy generiert das SELECT.

```python
class Transaction(Base):
    __tablename__ = "transactions"
    id:         Mapped[int]   = mapped_column(primary_key=True)
    account_id: Mapped[int]   = mapped_column(ForeignKey("accounts.id"))
    kind:       Mapped[str]
    amount:     Mapped[float]

    account: Mapped["Account"] = relationship(back_populates="transactions")

# Verwendung:
with Session(engine) as session:
    account = session.get(Account, 1)
    for tx in account.transactions:        # SELECT erzeugt SQLAlchemy selbst
        print(tx.kind, tx.amount)
```

Lab 16

Datenbanken mit SQLAlchemy